

## PARTE TEORICA EGB

### Modo protegido

#### 1) Mecanismos de Protección de acceso al Segmento en Modo Protegido:

A. Realice un diagrama completo que permita comprender el mecanismo de protección de memoria del 80x386 que permite un no acceso directo al segmento. Explique detalladamente como interpreta un valor en un registro de segmento si TI=1 o TI=0

#### Conceptos clave del mecanismo de protección de memoria

El 80386 usa **segmentación con protección**. Esto significa que cada segmento de memoria tiene un descriptor que define:

**Base:** Dirección inicial del segmento.

**Límite:** Tamaño máximo del segmento.

**Permisos de acceso:** Lectura, escritura, ejecución, privilegio.

El procesador **no accede directamente** a la memoria usando los registros de segmento. En su lugar:

1. El registro de segmento (CS, DS, ES, SS, FS, GS) **no contiene la dirección base directa**, sino un **selector de segmento**.
2. El **selector de segmento** apunta a un **descriptor** en una tabla de descriptores (GDT o LDT).

---

#### Selector de segmento y bit TI

El **selector de segmento** es un registro de 16 bits que tiene esta estructura:

| Bits | Nombre | Función |
|------|--------|---------|
|------|--------|---------|

|      |       |                                    |
|------|-------|------------------------------------|
| 15-3 | Index | Índice en la tabla de descriptores |
| 2    | TI    | Tabla de índice: 0 = GDT, 1 = LDT  |
| 1-0  | RPL   | Privilegio relativo al anillo 0-3  |

### Interpretación del bit TI:

Si **TI = 0** → Usa la **Global Descriptor Table (GDT)**.

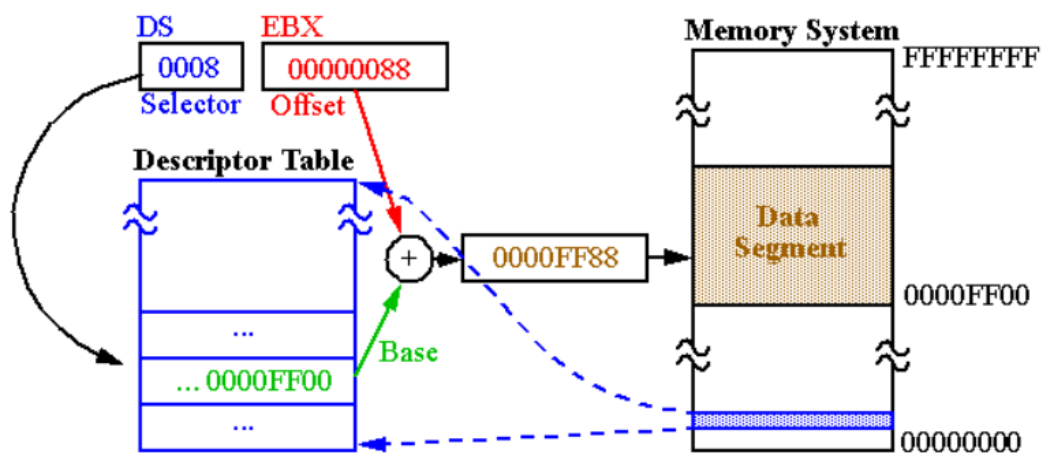
El índice apunta a un descriptor global, compartido por todos los procesos.

Si **TI = 1** → Usa la **Local Descriptor Table (LDT)** del proceso actual.

El índice apunta a un descriptor privado del proceso, lo que permite protección de memoria entre procesos.

Esto es lo que evita el acceso directo a la memoria: nunca se usa la dirección real del segmento, solo se interpreta a través de la tabla correspondiente.

## Protected Mode Memory Addressing



**Nota:** Registros de segmento del 80386

| Registro | Nombre                                      | Uso principal                                              | Descripción detallada                                                                                                                                                                                                                                                            |
|----------|---------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CS       | <i>Code Segment</i><br>(Segmento de código) | Contiene el <b>selector del segmento de código</b> actual. | Define la base y los límites del segmento que contiene las <b>instrucciones ejecutables</b> del programa. El procesador obtiene la próxima instrucción a ejecutar usando: Dirección lineal = Base(CS) + EIP.<br>Incluye el <b>nivel de privilegio actual (CPL)</b> del programa. |
| DS       | <i>Data Segment</i><br>(Segmento de datos)  | Segmento por defecto para                                  | Cualquier instrucción que acceda a memoria (como MOV AX, [BX]) usará por defecto la                                                                                                                                                                                              |

| Registro  | Nombre                                               | Uso principal                                                 | Descripción detallada                                                                                                                                                                                     |
|-----------|------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|           |                                                      | <b>acceder a variables o datos generales.</b>                 | base del segmento DS, a menos que se indique otro segmento.                                                                                                                                               |
| <b>SS</b> | <i>Stack Segment</i><br>(Segmento de pila)           | Segmento que contiene la <b>pila del programa</b> (stack).    | Las operaciones que usan la pila (como PUSH, POP, CALL, RET, interrupciones) usan SS y el puntero de pila (ESP o SP) para calcular la dirección efectiva: Base(SS) + ESP.                                 |
| <b>ES</b> | <i>Extra Segment</i><br>(Segmento extra)             | Segmento adicional de datos.                                  | Permite direccionar datos adicionales o realizar operaciones de movimiento de datos (por ejemplo, con las instrucciones de cadena MOVS, LODS, STOS, etc.).                                                |
| <b>FS</b> | <i>Extra Segment F</i><br>(Segmento extra adicional) | Segmento de datos <b>adicional</b> , introducido en el 80386. | Ofrece un espacio de segmentación adicional para estructuras especiales (por ejemplo, en sistemas operativos modernos, a veces se usa para apuntar al bloque de control del hilo o variables del kernel). |
| <b>GS</b> | <i>Extra Segment G</i><br>(Segmento extra adicional) | Otro segmento de datos adicional.                             | Similar a FS; se usa frecuentemente para propósitos específicos del sistema o del proceso (por ejemplo, en Linux x86, GS suele apuntar al Thread Local Storage – TLS).                                    |

## B. ¿Cuáles son las condiciones que deben cumplirse para que el selector correspondiente pueda acceder al segmento? ¿Cuál es la relación entre RPL y DPL?

Condiciones para que el Selector Acceda al Segmento

En el Modo Protegido, el registro de segmento ya no contiene directamente la dirección base, sino un **selector** que escoge un **descriptor** de una tabla (Global o Local). El descriptor contiene la posición, longitud límite y **derechos de acceso** del segmento de memoria.

Para que un selector acceda a un segmento, deben cumplirse varias condiciones relacionadas con la validez del descriptor, el límite del segmento y los niveles de privilegio.

### 1. Verificación del Selector y Descriptor

**1. Selección Válida del Descriptor:** El selector, ubicado en el registro de segmento, contiene un campo de 13 bits que selecciona uno de los 8192 descriptores de una de las tablas (Global o Local). El bit TI (indicador de tabla) especifica la tabla a usar: 0 para la Tabla de Descriptores Globales (GDT) y 1 para la Tabla de Descriptores Locales (LDT). El descriptor cero (descriptor nulo) no puede utilizarse para acceder a la memoria.

2. **Presencia del Segmento:** El descriptor debe indicar que el segmento está presente en la memoria física. Esto se verifica mediante el **bit P (Presente)** en el byte de derechos de acceso del descriptor. Si  $P=1$ , el segmento existe; si  $P=0$ , un intento de acceso generará una excepción 11 (o excepción 12 si se trata del segmento de pila).

3. **Límite del Segmento:** La dirección de desplazamiento (offset) utilizada por la instrucción debe estar dentro de la longitud permitida del segmento (definida por el campo Límite en el descriptor). Si el programa intenta acceder más allá de este límite, se interrumpe el programa del sistema operativo del microprocesador, indicando una falla de protección general (excepción 13).

## 2. Verificaciones de Privilegio

El acceso al segmento está gobernado por cuatro niveles jerárquicos de privilegio (PL, del 0 al 3), donde **el nivel 0 es el más privilegiado** (mayor confianza) y el nivel 3 es el menos privilegiado (menor confianza).

El acceso se concede a través de una comparación entre el nivel de privilegio de la tarea actual y el nivel de privilegio definido para el segmento:

- **Regla General (Acceso a Segmentos de Datos/Código):** Los datos almacenados en un segmento con nivel de privilegio “n” (0-3) solo pueden ser accedidos por código que se ejecuta a un nivel de privilegio por lo menos tan privilegiado como “n”.

- **Comparación de Niveles de Privilegio:** Cada vez que se carga un selector en un registro de segmento de datos (DS, ES), el microprocesador realiza validaciones de protección.

- La verificación se realiza comparando el **Nivel de Privilegio Actual (CPL)** con el **Nivel de Privilegio Efectivo (EPL)**.

- Si el EPL es **más privilegiado** que el CPL (es decir, si  $EPL < CPL$ , ya que menor número implica mayor privilegio), se genera una **excepción 13 (Violación general de protección)**.

- Por lo tanto, para que se conceda el acceso, se requiere que el CPL sea igual o más privilegiado que el EPL (es decir  $CPL \leq EPL$ ). Dado que  $EPL = \max(RPL, DPL)$  esto implica que  $CPL \leq DPL$  y  $CPL \leq RPL$ .

- **Segmento de Pila (SS) Específico:** Para cargar selectores en el registro SS, las reglas son más estrictas: el descriptor debe ser de un segmento de datos con habilitación de escritura, y tanto el **DPL** como el **RPL** deben ser **iguales al CPL**.

## 3. Permisos de Operación (Derechos de Acceso)

El **byte de derechos de acceso** dentro del descriptor controla cómo funciona el segmento.

- **Segmentos de Datos:** Se puede especificar si se puede escribir en el segmento (habilitación de escritura, Bit W) o si está protegido contra escritura. También se especifica la dirección de crecimiento (Bit ED).

- **Segmentos de Código:** El descriptor controla la ejecución y lectura. Por ejemplo, no se permite escribir sobre un segmento de código. Se puede especificar si se permite la lectura (Bit R). Un intento de leer o escribir donde no está permitido, o intentar ejecutar en un segmento de datos, genera una excepción 13.

## Relación entre RPL y DPL

**RPL (Nivel de Privilegio Solicitado)** y **DPL (Nivel de Privilegio del Descriptor)** son dos componentes cruciales utilizados en el sistema de protección jerárquico del modo protegido:

### 1. Definición de DPL (Descriptor Privilege Level):

- El DPL es el **nivel menos privilegiado** para el cual una tarea puede acceder a ese descriptor y al segmento asociado.
- Se establece mediante los bits 6 y 5 en el byte de derechos de acceso del descriptor.

### 2. Definición de RPL (Requestor Privilege Level):

- El RPL es el nivel de privilegio **solicitado** por la tarea que generó inicialmente este selector.
- Está contenido en los dos bits menos significativos del selector (los dos bits de menor peso).

### 3. Relación a través del EPL (Effective Privilege Level):

- El DPL y el RPL se utilizan juntos para calcular el **Nivel de Privilegio Efectivo (EPL)**.
- El EPL es el nivel **menos privilegiado** entre el RPL y el DPL.
- Debido a que un valor numérico menor indica mayor privilegio (00 es el más alto), el EPL se determina tomando el valor numéricamente **máximo** entre el RPL y el DPL:  $EPL = \max(RPL, DPL)$

### 4. Propósito del RPL:

- El RPL se utiliza para que un nivel de privilegio menos confiable que el Nivel de Privilegio Actual (CPL) pueda utilizar dicho selector.
- Su función principal es **verificar que los punteros** (selectores) pasados a un procedimiento del sistema operativo no accedan a datos que son de mayor privilegio que el procedimiento que originó el puntero.

En esencia, el DPL define la restricción de privilegio del segmento en sí mismo, mientras que el RPL define la restricción de privilegio del selector cuando se intenta realizar un acceso, asegurando que la verificación se realice al nivel menos privilegiado que permita la operación.

**C. Explique que son y para qué sirven los siguientes registros y bits: GDTR, GDTR, RPL, LDTR, bit G, bit D, bit P, bit S, bit E, bit A.**

Su consulta requiere la explicación del concepto y la función de varios registros de control y bits de atributo esenciales para el funcionamiento de la arquitectura 80x86 (especialmente 80286 y superiores) en el **Modo Protegido**.

A continuación, se detalla cada elemento, basándose en la información proporcionada en las fuentes:

#### Registros de Control del Sistema (Modo Protegido)

Los siguientes son registros especiales que contienen las direcciones y límites de las tablas de descriptores en memoria, y se consideran generalmente **invisibles para los programas de aplicación**.

#### GDTR (Registro de Tabla de Descriptores Globales)

- **Concepto y Función:** El GDTR (Global Descriptor Table Register) es un registro invisible para el programador que mantiene la **dirección lineal base de 32 bits** y el **límite de 16 bits** de la Tabla de Descriptores Globales (GDT).
- **Propósito:** La GDT contiene descriptores que definen segmentos que están disponibles para **todas las tareas** del sistema, como los segmentos de código y datos utilizados por el sistema operativo.
- **Gestión:** Cuando se desea la operación en Modo Protegido, la dirección y el límite de la GDT se cargan en el GDTR.

#### GDTL (Límite de la Tabla de Descriptores Globales)

- **Concepto y Función:** GDTL se refiere al componente de **Límite de 16 bits** almacenado dentro del registro GDTR.
- **Propósito:** Este límite define el tamaño máximo o la longitud de la Tabla de Descriptores Globales. Dado que el límite es de 16 bits, la longitud máxima de la tabla es de 64 Kbytes.

#### LDTR (Registro de Tabla de Descriptores Locales)

- **Concepto y Función:** El LDTR (Local Descriptor Table Register) es un registro de 16 bits que se utiliza para referenciar la Tabla de Descriptores Locales (LDT).
- **Propósito:** Las LDT contienen descriptores asociados a una tarea o aplicación particular, proporcionando un mecanismo para aislar los segmentos de código y datos de esa tarea de otras tareas y del sistema operativo.
- **Funcionamiento:** El LDTR se carga con un selector que apunta a un descriptor específico dentro de la GDT. Este descriptor define la dirección, el límite y los derechos de acceso de la LDT, y esta información se carga en la porción de caché invisible del LDTR.

#### RPL (Nivel de Privilegio Solicitado)

- **Concepto y Función:** El RPL (Requestor Privilege Level) es un campo de **dos bits** contenido en la porción menos significativa del **selector de segmento**.
- **Propósito:** El RPL especifica el **nivel de privilegio solicitado** para acceder a un segmento de memoria. Los niveles de privilegio van de 00 (más alto o más confiable) a 11 (más bajo o menos confiable).
- **Uso:** Se utiliza para que un nivel de privilegio menos confiable que el Nivel de Privilegio Actual (CPL) pueda usar el selector. Es crucial en la verificación de punteros para evitar que los procedimientos del sistema operativo accedan a datos de mayor privilegio que el procedimiento que originó el puntero. El nivel de privilegio efectivo (EPL) se calcula como el

valor numéricamente máximo (menos privilegiado) entre el RPL y el Nivel de Privilegio del Descriptor (DPL).

#### Bits de Atributo del Descriptor

Estos bits se encuentran dentro del descriptor de 8 bytes y definen las características operativas y de protección del segmento asociado.

#### Bit S (Bit Descriptor del Segmento)

- **Concepto y Función:** Es el bit 4 del byte de derechos de acceso. Su función es categorizar el descriptor en dos tipos generales.

- **Propósito:**

- Si **S = 1**: El descriptor define un segmento de **código o datos** (segmento de aplicación).
- Si **S = 0**: El descriptor define un segmento del **sistema** (estructuras de control como LDT, TSS o compuertas).

#### Bit P (Presente)

- **Concepto y Función:** Es el bit 7 del byte de derechos de acceso.

- **Propósito:** Indica si el segmento existe en la memoria física.

- Si **P = 1**: El segmento está presente en memoria.
- Si **P = 0**: El segmento no está en memoria. Un intento de acceso causa una excepción 11 (o 12 para el segmento de pila). Este mecanismo se utiliza para implementar la **memoria virtual**.

#### Bit E (Ejecutable)

- **Concepto y Función:** Es el bit 3 del byte de derechos de acceso, pero **solo es relevante si S = 1** (segmento de código o datos).

- **Propósito:** Determina la función principal del segmento:

- Si **E = 1**: El segmento es de **código** (contiene instrucciones).
- Si **E = 0**: El segmento es de **datos**.

#### Bit A (Accedido)

- **Concepto y Función:** Es el bit 0 del descriptor. Es manipulado directamente por el microprocesador.

- **Propósito:** Indica si el segmento ha sido referenciado recientemente.

- Si **A = 1**: El selector asociado ha sido cargado en un registro de segmento (CS, DS, etc.) o utilizado por una instrucción de prueba de selector.
- Si **A = 0**: El segmento no ha sido accedido.

#### Bit G (Granularidad)

- **Concepto y Función:** Es un bit adicional incluido en los descriptores a partir del **80386 y superiores**, no presente en el 80286.
- **Propósito:** Controla la interpretación del campo Límite del segmento:
  - Si **G = 0**: El límite se interpreta byte por byte. El límite especifica la longitud del segmento entre 1 byte y 1 Mbyte (FFFFFH).
  - Si **G = 1**: El valor del límite se multiplica por **4 Kbytes** (se le adjunta FFFH). Esto permite longitudes de segmento que van desde 4 Kbytes hasta **4 Gbytes**.

Bit D (Default/Big)

- **Concepto y Función:** Se encuentra en los descriptores del 80386 y superiores. Controla el tamaño predeterminado de los operandos y las direcciones de desplazamiento.
- **Propósito:**
  - Si **D = 0**: Se asume el modo de instrucciones de **16 bits** (o modo DOS). Las instrucciones utilizan direcciones de desplazamiento y registros de 16 bits de manera predeterminada (por ejemplo, utiliza IP en lugar de EIP).
  - Si **D = 1**: Se asume el modo de instrucciones de **32 bits**. Las direcciones de desplazamiento y los registros son de 32 bits por defecto (por ejemplo, utiliza EIP en lugar de IP, y ESP en lugar de SP para la pila)

**D. Práctica: Ejercicio de MP (Interpretación y funcionamiento) Consigna: Realice un diagrama interpretando en Modo Protegido el siguiente selector: CS=A3FF.**

La interpretación de un selector en **Modo Protegido** (disponible a partir del 80286 y superiores) es fundamentalmente diferente a la operación en Modo Real, ya que el registro de segmento (CS en este caso) contiene un **selector** que apunta a un **descriptor** en una tabla, en lugar de una dirección base directa.

El selector **CS = A3FFh** (donde CS es el registro de segmento de código) es un valor de 16 bits que se descompone en tres campos principales: el Índice, el Indicador de Tabla (TI) y el Nivel de Privilegio Solicitado (RPL).

A continuación, se presenta la interpretación de este selector mediante su descomposición y el diagrama de funcionamiento implícito en el Modo Protegido.

Interpretación del Selector: CS = A3FFh

Convertimos el valor hexadecimal **A3FFh** a binario para identificar sus componentes (16 bits):

## 1. Componentes del Selector

| Campo | Bits | Valor<br>(Binario) | Valor<br>(Decimal) | Significado en Modo Protegido |
|-------|------|--------------------|--------------------|-------------------------------|
|-------|------|--------------------|--------------------|-------------------------------|

|               |      |                     |                       |                                                                                                                                              |
|---------------|------|---------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RPL</b>    | 1:0  | 11                  | <b>3</b>              | <b>Nivel de Privilegio Solicitado (Requested Privilege Level).</b><br>Es el nivel de menor privilegio (el 3 es el más bajo/menos confiable). |
| <b>TI</b>     | 2    | 1                   | <b>1</b>              | <b>Indicador de Tabla (Table Indicator).</b> Si es 1, se utiliza la <b>Tabla de Descriptores Locales (LDT).</b>                              |
| <b>Índice</b> | 15:3 | 1010 0011<br>1111 1 | <b>1311</b><br>(51Fh) | <b>Número de Descriptor.</b> Este es el índice para seleccionar uno de los 8192 descriptores en la tabla especificada por TI.                |

#### Diagrama de Funcionamiento en Modo Protegido

El selector CS=A3FFh realiza el siguiente proceso para acceder al segmento de código correspondiente:

Paso 1: Localizar la Tabla de Descriptores

1. **TI = 1 (LDT):** El bit TI indica que el microprocesador debe utilizar la **Tabla de Descriptores Locales (LDT).**
2. **Registro LDTR:** La dirección base y el límite de la LDT se encuentran en el registro invisible para el programador llamado **LDTR** (Registro de Tabla de Descriptores Locales).

Paso 2: Seleccionar el Descriptor

1. **Índice (51Fh):** El índice, con valor 51Fh (1311 decimal), se utiliza para calcular la ubicación del descriptor dentro de la LDT.

◦ Dado que cada descriptor tiene **ocho bytes de longitud**, la dirección de memoria del descriptor se calcula como: **Dirección Base de LDT + (1311 \* 8).**

2. **Descriptor Obtenido:** El microprocesador recupera el descriptor de 8 bytes (Descriptor 1311) de la LDT.

Paso 3: Almacenar Información en la Caché de Segmento

1. **Carga en CS invisible:** Los 8 bytes del descriptor (que contienen la dirección base del segmento, su límite y los derechos de acceso) se cargan en la porción **invisible** (caché) del registro CS.

Paso 4: Verificaciones de Protección

Antes de permitir la ejecución del código, se llevan a cabo comprobaciones basadas en los niveles de privilegio:

1. **Verificación de Existencia (Bit P):** El bit **P (Presente)** del descriptor debe ser 1. Si P=0, el segmento no está en memoria física y se genera una excepción 11 (o 12 para SS).
2. **Verificación de Tipo de Segmento (Bit E):** Dado que CS es el registro de segmento de código, el bit ejecutable (E) del descriptor debe valer 1.
3. **Verificación de Privilegio:** Se verifica el acceso comparando los niveles de privilegio:
  - **RPL (3):** El Nivel de Privilegio Solicitado es 3 (el más bajo).

- **DPL (Descriptor Privilege Level):** Nivel de privilegio del segmento, encontrado en el descriptor.
- **CPL (Current Privilege Level):** Nivel de privilegio actual de la tarea, tomado de los dos bits menos significativos del registro CS (que es el RPL si se acaba de cargar, pero generalmente se toma como el CPL del segmento de código que se está ejecutando).

4. Para que se otorgue el acceso, el Nivel de Privilegio Actual (CPL) debe ser igual o más privilegiado que el **Nivel de Privilegio Efectivo (EPL)**, donde EPL es el nivel menos privilegiado (valor numéricamente mayor) entre RPL y DPL ( ). Si el EPL es más privilegiado que el CPL (es decir,  $EPL < CPL$ ), se genera una **excepción 13 (Violación General de Protección)**.

Paso 5: Generación de Dirección Lineal

1. **Dirección Base:** Una vez que las verificaciones son exitosas, la **Dirección Base de 32 bits** (o 24 bits en el 80286) se extrae del descriptor almacenado en la caché de CS.
2. **Dirección Lineal:** Esta Dirección Base se suma a la Dirección de Desplazamiento (Offset, típicamente EIP/IP para el segmento de código) para formar la Dirección Lineal.
3. **Acceso Final:** Esta Dirección Lineal, si no se utiliza paginación, se convierte en la Dirección Física que sale por el bus de direcciones.

-----  
**En analogía:**

Si el selector CS = A3FFh fuera una tarjeta de acceso a un edificio de alta seguridad, la interpretación sería la siguiente:

1. **Los bits RPL (11, nivel 3) y TI (1, LDT)** indican que el usuario tiene el **mínimo privilegio (3)** y está pidiendo acceso a una lista de recursos **locales** (LDT) de su área de trabajo.
2. **El Índice (51Fh)** le dice al guardia (el microprocesador) que mire el **registro 1311** de esa lista local.
3. El registro 1311 (el Descriptor) es el que contiene la **ubicación exacta** (Dirección Base) del recurso (Segmento de Código) y las **reglas de uso** (DPL, permisos de lectura/ejecución).
4. El guardia compara el nivel de privilegio de la tarjeta (RPL=3) con el nivel requerido por el recurso (DPL). Si el solicitante (la tarea actual, CPL) es de nivel 3 o menor (más privilegiado, ej. 2, 1, 0) que el nivel efectivo de la solicitud, se concede el acceso al recurso.

## **2) Descriptores de Segmento y Descriptores de Sistema:**

**A. ¿Cuántas tablas de descriptores existen en la arquitectura 80x386? Explique cada una de ellas. NOTA: No es lo mismo “tablas de descriptores” que “descriptores”.**

Basándose en la arquitectura del microprocesador **80386** (expansión IA32) existen **tres** tablas principales de descriptores que definen y gestionan los segmentos y las rutinas del sistema.

Estas tablas son necesarias para el esquema de direccionamiento en Modo Protegido, donde el registro de segmento ya no contiene una dirección base directa, sino un **selector** que apunta a un **descriptor** dentro de una de estas tablas. El descriptor, a su vez, contiene la posición, longitud y derechos de acceso del segmento de memoria.

A continuación, se detalla cada una de estas tablas:

#### 1. Tabla de Descriptores Globales (GDT)

1. **Propósito y Contenido:** La GDT (Global Descriptor Table) contiene descriptores que definen segmentos que **están disponibles para todas las tareas** del sistema. Generalmente, contiene los segmentos de código y datos utilizados por el sistema operativo, los segmentos de estado de tarea (TSS) y los descriptores para las Tablas de Descriptores Locales (LDT) utilizadas en el sistema.

2. **Alcance:** Los descriptores globales se aplican a todos los programas y a menudo se les denomina descriptores de sistema.

3. **Gestión:** Todos los sistemas basados en el 80286 en Modo Protegido contienen una GDT. La primera entrada de la GDT (descriptor cero) se nombra **descriptor nulo** y debe contener solo ceros, sin poder utilizarse para acceder a la memoria.

4. **Ubicación:** La GDT se localiza mediante el registro **GDTR** (Registro de Tabla de Descriptores Globales), el cual mantiene la dirección lineal base de 32 bits y el límite de 16 bits de la tabla.

5. **Tamaño:** La GDT es un arreglo de longitud variable que puede tener entre 8 y 65.536 bytes. Puede contener hasta 8192 descriptores, dado que cada descriptor es de ocho bytes de longitud.

#### 2. Tabla de Descriptores Locales (LDT)

1. **Propósito y Contenido:** Las LDT (Local Descriptor Table) contienen descriptores asociados a una **tarea o aplicación particular**. Su objetivo es aislar los segmentos de código y datos de una tarea dada del sistema operativo y de otras tareas.

2. **Alcance:** Los descriptores locales son, por lo general, únicos para una aplicación. Generalmente, los sistemas operativos están diseñados para que cada tarea tenga su propia LDT.

3. **Contenido Específico:** Solo puede contener descriptores de código, datos, pila, compuertas de tarea y compuertas de llamada.

4. **Ubicación:** La LDT se localiza mediante el registro **LDTR** (Registro de Tabla de Descriptores Locales). El LDTR se carga con un selector que, a su vez, accede a un descriptor almacenado en la GDT que define la LDT.

5. **Tamaño:** Al igual que la GDT, es un arreglo de longitud variable que puede tener entre 8 y 65.536 bytes y puede mantener hasta 8192 descriptores.

#### 3. Tabla de Descriptores de Interrupción (IDT)

1. **Propósito y Contenido:** La IDT (Interrupt Descriptor Table) es la tercera tabla esencial y se utiliza para manejar las interrupciones y excepciones en Modo Protegido.

2. **Función:** Contiene descriptores que apuntan a la ubicación de hasta **256 rutinas de servicio de interrupción** (*interrupt handlers*). Las entradas de la IDT son referenciadas por la instrucción INT, vectores externos de interrupción y excepciones (interrupciones internas del microprocesador).
  3. **Gestión:** Debe conocerse el valor máximo del tipo de interrupción que se va a utilizar para establecer correctamente el límite del registro asociado (IDTR).
  4. **Ubicación:** Se localiza mediante el registro **IDTR** (Registro de Tabla de Descriptor de Interrupciones), que guarda la dirección lineal base de 32 bits y el límite de 16 bits de la tabla.
  5. **Tamaño:** La IDT es una matriz de longitud variable, con una longitud máxima de 65.536 bytes, y puede mantener hasta 8192 descriptores.
- 

#### **Nota sobre Tablas vs. Descriptores:**

Es crucial distinguir entre las tablas y los descriptores:

- **Tabla de Descriptores:** Es la estructura en memoria (un *array*) que puede tener hasta 64 Kbytes de longitud y almacena múltiples descriptores (hasta 8192).
- **Descriptor:** Es una estructura de **ocho bytes** que contiene la información crucial (dirección base, límite, nivel de privilegio—DPL— y tipo de segmento) que el microprocesador utiliza para mapear una dirección lógica a una dirección lineal. El **selector** (contenido en el registro de segmento) es el que apunta a un descriptor específico dentro de una tabla.

**B. Explique de qué forma actúa el bit S como atributo de un descriptor. Utilice esta información para decir en qué tipo de descriptor se convierte el mismo. ¿Qué “tipos” de estos descriptores existen?**

El bit **S** (Bit Descriptor del Segmento) es un atributo clave dentro del **byte de derechos de acceso** de un descriptor en el Modo Protegido (arquitectura 80x286 y superiores). Su función principal es clasificar la naturaleza del segmento de memoria que el descriptor está describiendo, dividiéndolos en dos categorías principales.

Función del Bit S como Atributo de un Descriptor

El **bit S (Bit 4)** actúa como un indicador de la categoría general del segmento: si el segmento pertenece a una aplicación de usuario (código o datos) o si pertenece a estructuras de control del sistema operativo.

La actuación del bit S se define de la siguiente manera:

- **S = 1:** El descriptor define un segmento de **código o datos**. Estos segmentos están típicamente disponibles para el uso de programas de aplicación.
- **S = 0:** El descriptor define un segmento del **sistema**. Estos segmentos se utilizan para controlar y operar el sistema de memoria protegida y otras características, y suelen contener descriptores que definen tareas o estructuras de control.

## Tipos de Descriptor Resultantes

Dependiendo del valor del bit S, el descriptor se convierte en uno de los siguientes tipos generales:

### 1. Descriptor de Segmento de Código o Datos (S = 1)

Cuando S = 1, el descriptor es de un segmento que contiene información (datos, pila) o instrucciones (código). El tipo específico dentro de esta categoría se determina por el **Bit Ejecutable (E)** (Bit 3) en el mismo byte de derechos de acceso:

| Bit E<br>(Ejecutable) | Tipo de<br>Descriptor     | Atributos Clave Adicionales                                                                                                   |
|-----------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>E = 1</b>          | <b>Segmento de Código</b> | Define si el segmento es conforme (C) y si permite lectura (R).<br><b>No se permite escribir</b> sobre un segmento de código. |
| <b>E = 0</b>          | <b>Segmento de Datos</b>  | Define la dirección de expansión (ED) (hacia arriba o hacia abajo) y si permite escritura (W).                                |

### 2. Descriptor de Segmento del Sistema (S = 0)

Cuando S = 0, el descriptor se utiliza para estructuras internas del sistema. El tipo específico de descriptor del sistema está codificado en los **bits 3 a 0** del byte de derechos de acceso.

#### Tipos de Descriptores de Sistema (S = 0)

En el microprocesador 80286, y por extensión en el 80386 (que posee tipos adicionales no listados aquí), existen varios tipos de descriptores de sistema que el bit S=0 identifica.

Los tipos de descriptores de sistema disponibles en el 80286 (Bits 3-0 cuando S=0) incluyen:

- **Inválido (0000).**
- **TSS Disponible (0001):** Descriptor de Segmento de Estado de la Tarea (Task State Segment) disponible.
- **LDT (0010):** Descriptor de Tabla de Descriptores Locales.
- **TSS Ocupado (0011):** Descriptor de Segmento de Estado de la Tarea ocupado (es decir, en una cadena de tareas activas).
- **Compuerta de Llamada (0100):** Se utiliza para transferir el control del programa, especialmente para cambiar a un nivel más privilegiado.
- **Compuerta de Tarea (0101):** Se utiliza para realizar cambios de tarea.
- **Compuerta de Interrupción (0110):** Se utiliza para especificar rutinas de servicio de interrupción (deshabilita las interrupciones al activarse).
- **Compuerta de Trampa (0111):** Se utiliza para especificar rutinas de servicio de interrupción (no altera el indicador de interrupciones IF al activarse).

Es importante notar que el 80386 y los procesadores posteriores reservan ocho tipos adicionales para sus propias estructuras. Los descriptores de sistema, junto con los segmentos de código/datos, comparten atributos comunes como el bit Presente (P) y el Nivel de Privilegio del Descriptor (DPL).

### C. Explique el concepto y funcionamiento de un "Call via Gate"

El concepto de "**Call via Gate**" (**Llamada a través de Compuerta**), o *Compuerta de Llamada* (*Call Gate*), es un mecanismo esencial en la arquitectura x86 (a partir del 80286 y superiores) cuando opera en **Modo Protegido**. Este mecanismo forma parte del sistema de protección jerárquica que utiliza los Niveles de Privilegio (PL: Privilege Level).

#### Concepto y Propósito

El concepto de una compuerta de llamada se centra en el **control seguro y la transferencia de control entre segmentos**, especialmente cuando esta transferencia implica un **cambio en el Nivel de Privilegio (PL)**.

1. **Indirección y Control:** Las compuertas son descriptores de sistema (donde el bit S del descriptor vale 0). Proporcionan un nivel de **indirección** entre el origen de la transferencia y el destino.
2. **Transferencia de Privilegio:** Uno de los usos más importantes de las compuertas de llamada es proporcionar un método seguro para realizar **transferencias a un nivel de privilegio más privilegiado** (es decir, a un nivel numéricamente menor, como de la aplicación PL=3 al núcleo PL=0).
3. **Control de Acceso:** Permiten que los diseñadores del sistema operativo controlen los **puntos de entrada** a las rutinas del sistema operativo (por ejemplo, funciones que realizan manejo de memoria u operaciones de entrada/salida confiables). Un intento de acceder a otro lugar (que no sea el punto de entrada definido por la compuerta) causará una excepción 13 (Violación General de Protección).
4. **Verificación de Protección:** La indirección que ofrecen las compuertas permite al procesador realizar **verificaciones de protección automáticamente** antes de transferir el control.

#### Funcionamiento de un "Call via Gate"

Una llamada a través de compuerta se invoca mediante una instrucción **CALL** intersegmento, la cual es sintácticamente idéntica a una llamada a subrutina normal, pero cuyo operando es un selector que apunta a un descriptor de compuerta en la Tabla de Descriptores Globales (GDT) o Local (LDT).

#### Estructura del Descriptor de Compuerta

El descriptor de compuerta de llamada (tipo 0100) tiene un formato específico que contiene tres campos clave, además de los derechos de acceso:

1. **Selector y Offset:** Estos dos campos forman un **puntero largo** (selector y *offset* de 16 bits) que apunta al inicio de la rutina de código objetivo.
2. **Cantidad de Palabras (*Word Count*):** Este campo especifica **cuántos parámetros** (en palabras de 16 bits) deben copiarse de la pila de la rutina llamadora a la pila de la rutina llamada. Este campo solo se utiliza si hay un cambio de Nivel de Privilegio (PL).

## Reglas de Privilegio para la Transferencia

Para que la transferencia a través de una compuerta de llamada se realice con éxito, deben cumplirse rigurosas reglas de privilegio:

- **Acceso a la Compuerta:** Una tarea puede acceder a una compuerta solo si el **Nivel de Privilegio Efectivo (EPL)** de la solicitud es **igual o más privilegiado** que el **Nivel de Privilegio del Descriptor (DPL)** de la compuerta.
- **Regla de Transferencia:** Las compuertas de llamada solo pueden transferir el control a un **nivel más privilegiado**.
- **Privilegio del Selector y Tarea Actual:** El RPL (Nivel de Privilegio Solicitado) en el selector que apunta a la compuerta y el CPL (Nivel de Privilegio Actual) de la tarea deben tener un privilegio igual o mayor que el DPL de la compuerta.
- **Privilegio del Segmento de Código Objetivo:** El segmento de código seleccionado dentro de la compuerta debe tener un privilegio igual o mayor que el CPL de la tarea que realiza la llamada.

## Mecanismo de Cambio de Pila (Cambio de Privilegio)

Si la llamada requiere una transición a un nivel de privilegio más alto (por ejemplo, de PL=3 a PL=0), el sistema debe cambiar a una **pila más privilegiada** (la pila del núcleo).

Cuando se activa una llamada que cambia el nivel de privilegio, el microprocesador realiza automáticamente las siguientes acciones:

1. **Carga del Destino:** Carga el **CS:IP** de la compuerta (el segmento de código de la rutina de destino) y verifica su validez.
2. **Apuntadores de Pila Anteriores:** Empuja (guarda) el **viejo SS** (Segmento de Pila) en la nueva pila.
3. **Apuntadores de Pila Anteriores:** Empuja (guarda) el **viejo SP** (Puntero de Pila) en la nueva pila.
4. **Copia de Parámetros:** Copia la cantidad de palabras de 16 bits especificadas en el campo de *Cantidad de Palabras* del descriptor de compuerta, desde la pila anterior a la **nueva pila** (la pila del nivel privilegiado).
5. **Dirección de Retorno:** Empuja la dirección de retorno (el viejo CS:IP de la tarea llamadora) en la pila.

Este cambio de pilas y copia de parámetros asegura que la rutina de más alto privilegio (el *handler*) pueda acceder a los parámetros de manera controlada y segura, y que el contexto de retorno se mantenga intacto para restaurar la ejecución al nivel de privilegio original (utilizando la instrucción RET intersegmento).

### 3) Segmento de Estado de la Tarea (TSS):

A .¿Qué se busca con el uso del TSS relacionado al uso del procesador y la conmutación de tareas?

El **Segmento de Estado de la Tarea (TSS)**, o *Task State Segment*, es un componente crucial en el **Modo Protegido** de los microprocesadores 80286 y superiores (IA32). Es un **segmento especial con formato fijo** que contiene toda la información sobre el estado de una tarea.

El uso del TSS, en relación con el procesador y la conmutación de tareas, busca principalmente lograr un **cambio de contexto rápido, ordenado y seguro** para soportar sistemas operativos multitarea/multiusuario.

A continuación, se explica lo que se busca con el uso del TSS:

#### 1. Habilitar la Multitarea y la Conmutación de Tareas Rápida

El objetivo primario del TSS es facilitar la capacidad del sistema operativo para **cambiar rápidamente entre tareas o procesos**.

- **Soporte de Hardware:** El microprocesador, como el 80286, incluye una **instrucción de cambio de tarea en el hardware**. Esto permite que el procesador conmute entre varias tareas en un tiempo considerablemente corto, específicamente en menos de **20 microsegundos** en el 80286, o aproximadamente **17 µs**.

- **Mecanismo Simple y Ordenado:** La conmutación de tareas se invoca ejecutando una instrucción **JMP** o **CALL intersegmento** que referencia a un descriptor de TSS o una compuerta de tarea en la GDT o LDT. Este proceso permite que los sistemas de multitarea conmuten de una tarea a otra de una **forma simple y ordenada**.

#### 2. Guardado y Restauración del Estado Completo (Context Switching)

El TSS encapsula el **estado completo de la máquina** para una tarea específica, permitiendo suspender y reanudar tareas de manera transparente.

- **Salvado del Estado:** Cuando ocurre un cambio de tarea, el procesador utiliza el TSS para **salvar el estado entero de la máquina**. Esto incluye todos los registros de propósito general (como AX, CX, DX), registros de segmento, el espacio de direcciones, el registro de indicadores (Flags) y un puntero a la tarea anterior.

- **Carga del Nuevo Estado:** Inmediatamente después de salvar el contexto de la tarea saliente en su TSS, el procesador **carga un nuevo estado de ejecución** extrayendo la información del TSS de la tarea entrante (la cual se identifica mediante el registro **TR**, *Task Register*).

- **Retorno de Tarea:** Cuando se ejecuta la instrucción **IRET**, el control retorna a la tarea que había sido interrumpida. El estado de la tarea que se está ejecutando se almacena en el TSS y el estado de la vieja tarea se restaura de su propio TSS.

#### 3. Soporte para Estructuras de Control y Privilegio

El TSS es fundamental para la operación de protección y gestión del sistema:

- **Punteros de Pila Privilegiados:** El TSS contiene los punteros de pila (**SS:SP**) necesarios para la tarea en los diferentes **Niveles de Privilegio** (CPL = 0, 1 y 2). Esto es esencial para realizar

llamadas de procedimientos que cambian el nivel de privilegio, asegurando que el *kernel* utilice su propia pila segura.

- **Verificación de Protección:** Durante la operación de cambio de tarea, el hardware realiza **verificaciones de protección** de manera automática.
- **Información Adicional del Sistema Operativo:** Si bien el formato del TSS es fijo para la arquitectura, el límite del segmento puede ser mayor que el tamaño mínimo. Este espacio adicional permite al sistema operativo almacenar información extra sobre la tarea, como por qué está inactiva, el tiempo de ejecución o los archivos abiertos.
- **Control del Coprocesador (TS Flag):** El TSS, en conjunto con el bit **TS (Task Switched)** en el registro de control CRO, ayuda a gestionar el estado del coprocesador. Si el procesador detecta un cambio de tarea (TS se pone a 1), se puede generar una excepción para que el manejador decida si necesita salvar el contexto del coprocesador para la tarea saliente.

## **B. Explique el funcionamiento y realice un diagrama de interpretación del mismo. Esquema del TSS.**

El **Segmento de Estado de la Tarea (TSS)** es una estructura de datos esencial en la arquitectura 80x86 (a partir del 80286) para el manejo de la multitarea y la protección de los recursos en el **Modo Protegido**.

Funcionamiento del Segmento de Estado de la Tarea (TSS)

El TSS es un **segmento especial con formato fijo** que contiene toda la información necesaria para describir el estado completo de una tarea en ejecución.

### **1. Conmutación de Tareas (Task Switching)**

El principal propósito del TSS es permitir la **conmutación de tareas (cambio de contexto)** en el hardware de manera eficiente, lo cual es fundamental para los sistemas operativos multitarea/multiusuario.

- **Activación:** La operación de cambio de tareas puede ser invocada automáticamente por el microprocesador mediante la ejecución de una instrucción **JMP** o **CALL intersegmento** que referencia a un descriptor de TSS o a una compuerta de tarea. También puede ser activada por una interrupción, excepción o trampa si hay un descriptor de compuerta de tarea en la Tabla de Descriptores de Interrupción (IDT).
- **Ahorro de Estado:** Cuando se realiza un cambio de tarea, el procesador utiliza el TSS para **salvar el estado entero de la máquina** (es decir, el contexto de la tarea saliente). Esto incluye los registros de propósito general, los registros de segmento, el puntero a la tarea anterior y el registro de indicadores (*Flags*).
- **Carga del Nuevo Estado:** Posteriormente, el procesador **carga un nuevo estado de ejecución** (el contexto de la tarea entrante) extrayendo la información del TSS de la nueva tarea.
- **Tiempo de Operación:** Esta instrucción de cambio de tarea en el *hardware* permite al microprocesador conmutar entre varias tareas en un tiempo considerablemente corto, estimado en aproximadamente **17 µs** en el 80286.

## 2. Gestión de Privilegios y Pilas

El TSS contiene información crítica para mantener la protección y la integridad del sistema:

- **Punteros de Pila Privilegiados:** El TSS almacena los punteros de pila (SS:SP) que debe usar la tarea cuando opera en los diferentes niveles de privilegio del sistema (PL=0, PL=1 y PL=2). Si una llamada (por ejemplo, a través de una compuerta de llamada) requiere una transferencia a un nivel de privilegio más alto, el sistema cambia a la pila más privilegiada especificada en el TSS.

- **Identificación:** El **Registro de Tarea (TR)** apunta al TSS que describe la tarea en ejecución en ese momento, guardando un selector que referencia el descriptor de TSS en la Tabla de Descriptores Globales (GDT).

- **Retorno de Tarea:** La instrucción **IRET** (Retorno de Interrupción) puede utilizar el TSS para realizar una operación de cambio de tarea si el bit **NT (Tarea Anidada)** del registro de indicadores vale 1, permitiendo volver a la tarea que había sido interrumpida.

Esquema y Contenido del TSS (Segmento de Estado de la Tarea)

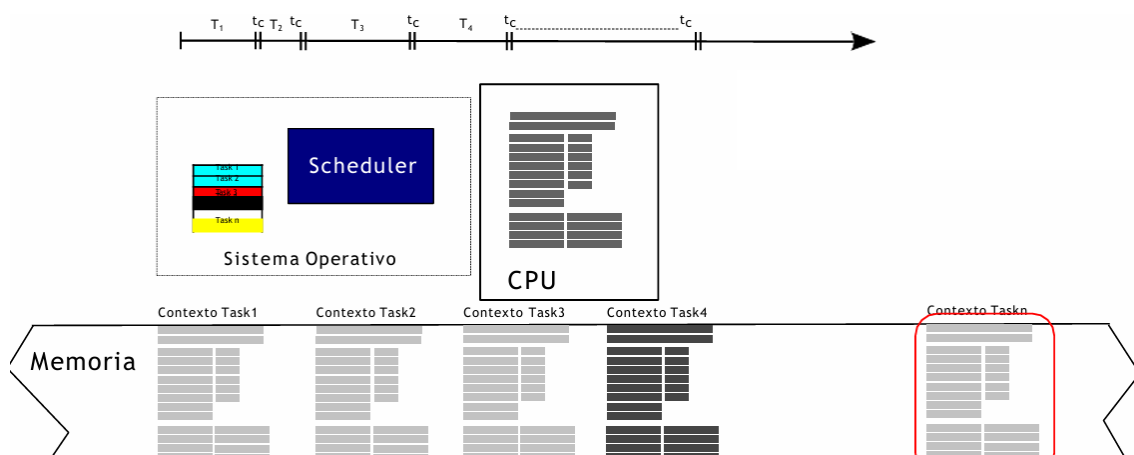
El TSS es un segmento de formato fijo. En el 80286, la longitud mínima del segmento debe ser mayor o igual a (44 bytes). El microprocesador 80386 utiliza un formato similar (TSS tipo 286) y también introduce el **TSS tipo 386** que incorpora registros de 32 bits y el **Mapa de Bits de Permisos de Entrada/Salida** para protección de E/S.

A continuación, se presenta el esquema del contenido del TSS, basado en el formato del 80286, que establece la base de la estructura:

| Offset<br>(Hexadecimal) | Bytes | Contenido (Estado<br>de la Tarea)            | Descripción                                                                                 |
|-------------------------|-------|----------------------------------------------|---------------------------------------------------------------------------------------------|
| 00h                     | 2     | <b>Puntero selector<br/>del TSS anterior</b> | Puntero a la TSS de la tarea que fue interrumpida (usado si NT=1).                          |
| 02h                     | 2     | <b>SP para CPL = 0</b>                       | Puntero de Pila (Stack Pointer) para el nivel de privilegio 0.                              |
| 04h                     | 2     | <b>SS para CPL = 0</b>                       | Segmento de Pila (Stack Segment) para el nivel de privilegio 0.                             |
| 06h                     | 2     | <b>SP para CPL = 1</b>                       | Puntero de Pila (Stack Pointer) para el nivel de privilegio 1.                              |
| 08h                     | 2     | <b>SS para CPL = 1</b>                       | Segmento de Pila (Stack Segment) para el nivel de privilegio 1.                             |
| 0Ah                     | 2     | <b>SP para CPL = 2</b>                       | Puntero de Pila (Stack Pointer) para el nivel de privilegio 2.                              |
| 0Ch                     | 2     | <b>SS para CPL = 2</b>                       | Segmento de Pila (Stack Segment) para el nivel de privilegio 2.                             |
| 0Eh                     | 2     | <b>IP (Punto de<br/>entrada)</b>             | Puntero de Instrucciones ( <i>Instruction Pointer</i> ) (siguiente instrucción a ejecutar). |

|     |   |                                 |                                                                                                                               |
|-----|---|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 10h | 2 | <b>Flags</b>                    | Registro de Indicadores de 16 bits.                                                                                           |
| 12h | 2 | <b>Registro AX</b>              | Registro Acumulador.                                                                                                          |
| 14h | 2 | <b>Registro CX</b>              | Registro Contador.                                                                                                            |
| 16h | 2 | <b>Registro DX</b>              | Registro de Datos.                                                                                                            |
| 18h | 2 | <b>Registro BX</b>              | Registro Base.                                                                                                                |
| 1Ah | 2 | <b>Registro SP</b>              | Puntero de Pila actual de la tarea.                                                                                           |
| 1Ch | 2 | <b>Registro BP</b>              | Puntero Base.                                                                                                                 |
| 1Eh | 2 | <b>Registro SI</b>              | Índice de Origen.                                                                                                             |
| 20h | 2 | <b>Registro DI</b>              | Índice de Destino.                                                                                                            |
| 22h | 2 | <b>Selector ES</b>              | Segmento Extra.                                                                                                               |
| 24h | 2 | <b>Selector CS</b>              | Segmento de Código.                                                                                                           |
| 26h | 2 | <b>Selector SS</b>              | Segmento de Pila.                                                                                                             |
| 28h | 2 | <b>Selector DS</b>              | Segmento de Datos.                                                                                                            |
| 2Ah | 2 | <b>Selector LDT de la tarea</b> | Selector que apunta a la Tabla de Descriptores Locales (LDT) de esta tarea.                                                   |
| 2Ch | N | <b>Disponible</b>               | Espacio adicional que el sistema operativo puede usar para almacenar información extra (si el límite del TSS es mayor a 2Ch). |

**Nota:** La información almacenada en el TSS mediante el cambio de tarea es específica de la tarea. En las versiones del 80386 y posteriores, los campos de registro se extienden a 32 bits (EIP, EAX, etc.) y se añade el Mapa de Bits de Permisos de E/S, lo que resulta en un TSS más grande.



#### 4) Paginado:

**A. Explique conceptualmente para qué sirve el proceso de paginado y como se relaciona con el sistema de segmentado.**

El proceso de paginación (paging) y su relación con el sistema de segmentación son características fundamentales de la Unidad de Manejo de Memoria (MMU) en el microprocesador 80386 y arquitecturas superiores (IA32) que operan en Modo Protegido.

Concepto y Servicio del Proceso de Paginación

La paginación es un tipo de manejo de memoria diseñado para sistemas operativos multitarea que gestionan **memoria virtual**. A diferencia de la segmentación, que modulariza programas y datos en segmentos de longitud variable, la paginación divide los programas en varias **páginas de tamaño fijo**.

El concepto y el servicio principal de la paginación son:

1. **Implementación de Memoria Virtual:** El proceso de paginación es esencial para implementar un sistema de memoria virtual, que permite que el programa tenga un tamaño mayor que la memoria física.
2. **Gestión del Espacio de Direcciones Físicas:** El mecanismo de paginación opera por debajo del segmentado y es transparente a este, permitiendo el **manejo del espacio de direcciones físicas**.
3. **Unidad de Tamaño Fijo:** La paginación divide los segmentos en páginas de **4 kilobytes (4KB)**. Este tamaño uniforme para todos los elementos simplifica el manejo de memoria y ayuda a evitar la fragmentación.
4. **Recuperación de Páginas Faltantes:** Si una página determinada que se necesita no se encuentra en la memoria física (un **fallo de página**), el 80386 indica esta situación al sistema operativo mediante la **excepción 14**. El sistema operativo entonces carga la página requerida desde el disco duro, y la ejecución del programa puede seguir, lo cual es transparente para la aplicación.
5. **Optimización Multitarea:** Debido al efecto de localidad (la mayoría de los programas utilizan referencias cercanas), solo una pequeña cantidad de páginas por tarea es necesaria en la memoria en un momento dado.

La paginación se habilita en el 80386 mediante el **bit PG (Paging Enable)** en el registro de control **CR0**.

Relación entre el Paginado y el Sistema de Segmentación

En el microprocesador 80386, la Unidad de Manejo de Memoria (MMU) está compuesta por una unidad de segmentación y una unidad de paginación. Estos dos sistemas trabajan en secuencia para traducir una dirección lógica en una dirección física final:

1. Segmentación: Traducción de Lógica a Lineal

- **Propósito de Segmentación:** La segmentación es el método que provee la base para la protección. Permite la gestión del **espacio de direcciones lógicas** al agregar un componente de

direccionamiento extra (el selector). Además, modulariza programas y datos en segmentos de longitud variable, que pueden tener un tamaño máximo de 4 gigabytes en el 80386.

- **Proceso:** El sistema de segmentación toma la dirección lógica (compuesta por el selector y el desplazamiento/offset/dirección efectiva) y, utilizando el descriptor del segmento, calcula la **Dirección Lineal de 32 bits**.

## 2. Paginación: Traducción de Lineal a Física

- **Interconexión:** El mecanismo de paginado opera **por debajo** y es **transparente al proceso de segmentación**.

- **Input/Output:** La unidad de paginación recibe la **dirección lineal de 32 bits** que proviene de la unidad de segmentación.

- **Proceso:** Si el paginado está habilitado (PG=1 en CR0), la unidad de paginación utiliza dos niveles de tablas (Directorio de Páginas y Tablas de Páginas) para traducir la dirección lineal en la **Dirección Física**. Esta dirección física es la que finalmente sale por el bus de direcciones.

En resumen, el sistema opera en dos fases de traducción obligatorias en el 80386 cuando se utiliza paginación:

### 1. Dirección Lógica (Segmento:Offset) Dirección Lineal (Segmentación)

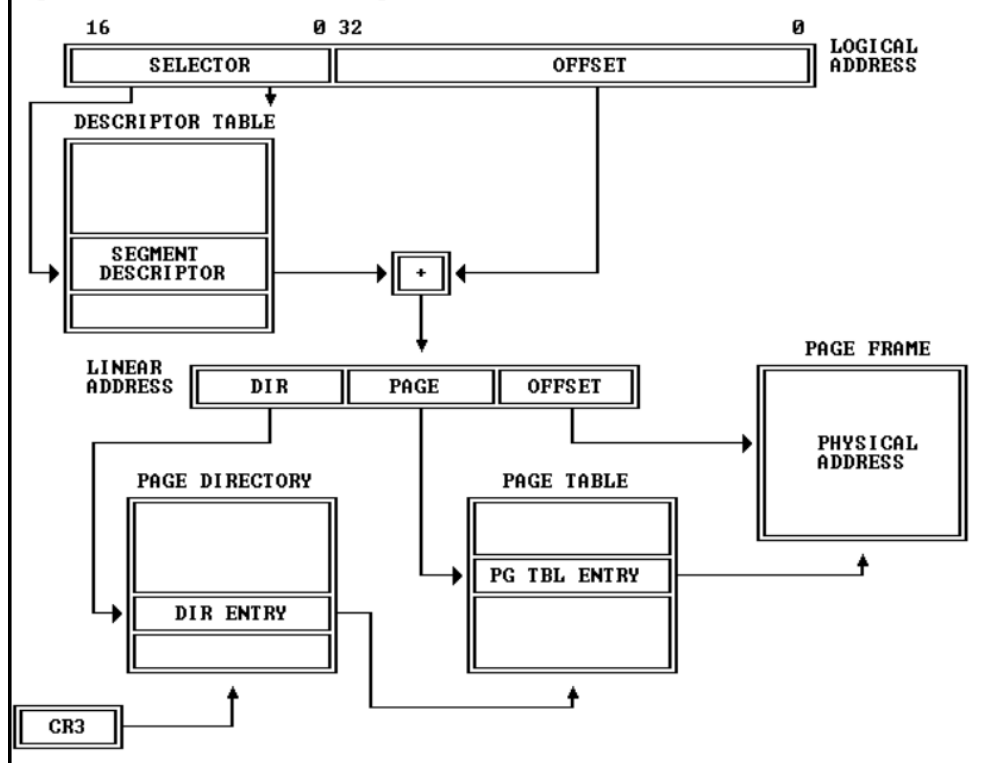
### 2. Dirección Lineal Dirección Física (Paginación)

Si la unidad de paginación no está habilitada (PG=0), la Dirección Lineal coincide con la Dirección Física. La paginación toma la gran estructura de longitud variable definida por la segmentación (el segmento) y la subdivide en unidades de tamaño fijo (páginas de 4KB) para su gestión en la memoria física.

### Analogía:

El proceso de direccionamiento es como un servicio de correo en dos etapas. La **Segmentación** actúa como la oficina de correos que recibe una **dirección lógica** (el nombre de una ciudad grande y un distrito, que es el Segmento), y la convierte en una **dirección lineal** más precisa (un número de edificio exacto dentro del territorio, que es la Base del Segmento sumada al Offset). Luego, la **Paginación** toma esa dirección lineal precisa (el número del edificio) y, consultando sus tablas (el directorio y las tablas de páginas), determina dónde se encuentra físicamente ese edificio en el espacio de almacenamiento real (la **Dirección Física**), permitiendo además traer partes del edificio (páginas) desde un almacenamiento secundario si no están presentes.

Figure 5-12. 80386 Addressing Mechanism



## B. ¿Qué hace la unidad de paginado? ¿Cómo lo hace?

La unidad de paginación es un componente fundamental de la **Unidad de Manejo de Memoria (MMU)** presente en el microprocesador **80386** y arquitecturas posteriores.

La paginación opera en el **Modo Protegido** y su mecanismo es **transparente al proceso de segmentación**.

¿Qué hace la unidad de paginado?

El proceso de paginación tiene tres propósitos conceptuales principales:

1. **Manejo del Espacio de Direcciones Físicas:** La paginación permite el manejo del espacio de direcciones físicas del sistema.
2. **Soporte de Memoria Virtual:** Es un tipo de manejo de memoria útil para sistemas operativos multitarea que gestionan **memoria virtual**. Permite que el programa tenga un tamaño mayor que la memoria física y solo requiere que una pequeña cantidad de páginas por tarea esté presente en la memoria en un momento dado, debido al efecto de localidad.
3. **División en Unidades Fijas:** La paginación divide los segmentos de longitud variable en varias **páginas de tamaño fijo**, típicamente de **4 kilobytes (4KB)**. Un tamaño uniforme para todos los elementos simplifica el manejo de memoria y ayuda a evitar la fragmentación.

Si una página determinada no se encuentra en la memoria, el 80386 lo indica al sistema operativo mediante la **excepción 14** (Fallo de Página), permitiendo que el sistema operativo cargue la página desde el disco rígido y continúe la ejecución, siendo este proceso transparente para la aplicación.

¿Cómo lo hace?

La unidad de paginación actúa como una segunda capa de traducción de direcciones en el Modo Protegido, tomando la dirección de salida de la segmentación y convirtiéndola en una dirección física.

#### 1. Habilitación y Entrada

Para que la unidad de paginación funcione, debe estar habilitada. Esto se logra poniendo el bit **PG (Paging Enable)** a uno en el registro de control **CR0**.

La unidad de paginación recibe una **dirección lineal de 32 bits** procedente de la unidad de segmentación.

#### 2. Tablas de Traducción (Dos Niveles)

El 80386 utiliza un mecanismo de dos niveles de tablas para traducir la dirección lineal en una dirección física. Los tres componentes de este mecanismo son: el **Directorio de Páginas**, las **Tablas de Páginas** y las **Páginas** mismas. Cada uno de estos elementos ocupa **4KB** en la memoria física.

Los registros de control involucrados son:

- **CR3 (Registro de Control):** Contiene la **dirección física inicial del Directorio de Páginas**.
- **CR2 (Registro de Control):** Mantiene la **dirección lineal de 32 bits** que causó el último fallo de página detectado.

#### 3. Proceso de Traducción

La dirección lineal de 32 bits se divide en tres campos que se utilizan secuencialmente para buscar la dirección física:

1. **Directorio de Páginas (Bits 31-22):** Los 10 bits más significativos de la dirección lineal (bits 31-22) se utilizan para seleccionar una de las 1024 entradas en el Directorio de Páginas. Esta entrada contiene la **dirección inicial de la siguiente tabla, la Tabla de Páginas**.
2. **Tabla de Páginas (Bits 21-12):** Los siguientes 10 bits de la dirección lineal (bits 21-12) seleccionan una de las 1024 entradas dentro de la Tabla de Páginas. Esta entrada contiene la **dirección inicial de la página**.
3. **Página/Offset (Bits 11-0):** Los 12 bits menos significativos de la dirección lineal (el *offset*) se concatenan con los 20 bits más significativos de la dirección de la página (obtenidos de la Tabla de Páginas) para formar la **Dirección Física** de 32 bits. Esta dirección física es colocada en el bus de direcciones.

#### 4. Optimización mediante TLB

Dado que el esquema de paginación requiere típicamente dos accesos a memoria (uno para el Directorio y otro para la Tabla de Páginas) por cada referencia a memoria solicitada por el software, el rendimiento se vería afectado.

Para mitigar esto, el 80386 utiliza un caché interno llamado **Buffer de Conversión por Búsqueda (TLB - Translation Lookaside Buffer)**.

- El **TLB** es un caché asociativo de cuatro vías que almacena **32 entradas de tablas de páginas** de uso más reciente.
- Cuando la unidad de paginación recibe una dirección lineal, primero compara los 20 bits más significativos de la dirección con las entradas del TLB.
- Si la entrada se encuentra en el caché (*cache hit*), se calcula inmediatamente la dirección física de 32 bits y se coloca en el bus de direcciones, sin necesidad de acceder a las tablas en la memoria RAM.

## 5. Manejo de Fallos de Página

Si la entrada de la tabla de páginas no se encuentra en el TLB (*cache miss*), o si una entrada de tabla indica que la página no está presente en memoria (bit **P = 0** en las entradas del directorio o la tabla), o si hay una violación de protección a nivel de página, el procesador genera una **excepción 14 (Fallo de Página)**. El registro **CR2** almacena la dirección lineal que causó dicho fallo.

El proceso de paginación, con su sistema de dos niveles y el uso del TLB, funciona como una **tabla de contenidos** sumamente detallada para la memoria. La segmentación le da al programa la **dirección lógica** de un capítulo (el segmento), y la paginación (utilizando el Directorio y las Tablas) encuentra la **ubicación física** exacta de cada frase (la página) que compone ese capítulo.

